

Fehlersuche

Möglichkeiten

1. Setze einen Haltepunkt an der interessanten Stelle (d.h. du musst dir sicher sein, dass vorher alles stimmt. Kontrolliere die Werte der Variablen im Debugger.
2. Gib in der Konsole Meldungen aus, wenn du nicht sicher bist, dass das Programm überhaupt an die richtige Stelle kommt:

   z.B. `System.out.println("Stelle 1 wurde erreicht.");`
3. Gib in der Konsole Werte von Variablen aus, wenn in einer Schleife nicht klar ist, ob die Berechnungen korrekt ausgeführt werden.

   z.B. `System.out.println("x ist "+x+ ", y ist "+y);`

Weitere Array-Algorithmen:

Lade die Vorlage `alg14a_zeitungsmeldung`


Breaking News

Klasse 9a schlägt 9b

Beim Binärspiel 123 Punkte mehr erreicht

Die 31 Schülerinnen und Schüler der Klasse 9a erreichten am 21. Nov. 2018 insgesamt 6450 Punkte in 72 Spielen. Das sind im Schnitt 89,53 Punkte.

Dagegen konnten die 27 Schülerinnen und Schüler der Klasse 9b nur 6327 Punkte in 40 Spielen erreichen. Damit hat die 9a den Klassenwettbewerb dieses Jahr gewonnen.

Implementation:

1. *Um den Sieger des Klassenwettbewerbs zu ermitteln, wurde eine Methode implementiert. Leider enthält sie zwei Fehler. Finde diese Fehler in der Methode `berechneSumme`, indem du eine der Möglichkeiten zur Fehlersuche anwendest.*
2. *Die Anzahl der Spiele wird auch noch mit 0 ausgegeben. Überlege dir, wie du die Anzahl der Spiele ermitteln kannst und setze die Variable in der Methode `schreibeZeitungsmeldung()` richtig.*
3. *Die Klasse 9b fühlt sich ungerecht behandelt, da sie der Meinung ist, im Durchschnitt viel besser gewesen zu sein. Um die Behauptung bestätigen zu können, musst du eine Methode `double berechneDurchschnitt(int[] zahlen)` implementieren, die den Durchschnitt aller Werte bestimmt und zurückgibt. Speichere den Wert in der vorgesehenen Variablen und ersetze das "XX" in der Textausgabe durch die Variable. Tipp: Verwende die Methode aus Aufgabe 1, um die Summe zu berechnen und teile diese Summe durch die Anzahl der summierten Werte.*
4. **** Implementiere eine Methode `int anzahlSpieler(String[] namen)`, die bestimmt, wie viele verschiedene Spieler am Spiel teilgenommen haben (es gibt keine zwei Spieler mit gleichem Namen, aber jeder Spieler kann mehrfach gespielt haben). Vergleiche dazu jeden Spielernamen mit den vorangegangenen. Nur wenn es ein neuer ist, darf er gezählt werden. Man muss sich also in einer boolean-Variable merken, wenn der Name schon vorkam.*
   Tipp: Zwei String vergleicht man so:

   ```
   if (string1.equals(string2)) {... }
   ```



Highscore-Tabelle

```
1: public void wastutes() {
2:  for (akt=0; akt+1 < zahlen.length; akt++) {
3:    if (zahlen[akt+1]> zahlen[akt]) {
4:       int dummy = zahlen[akt];
5:       zahlen[akt] = zahlen[akt+1];
6:       zahlen[akt+1] = dummy;
7:     }
8:   }
9: }
```



Gegeben ist folgendes Array. Der obige Algorithmus wird darauf angewendet.

i	0	1	2	3	4	5
zahlen[i]	45	23	21	12	34	56

1. *Gib an, welchen Wert die Arrayzellen haben, wenn akt = 3 ist und die Zeilen 4-6 ausgeführt werden. Zur Verdeutlichung kannst du dir das Video "Kluge Vögel - Die Werkzeugmacher" (https://www.youtube.com/watch?v=bvWWpahwMOc von Zeitpunt 1:16 - 2:18 Min) anschauen. Beschreibe die Auswirkung der drei Zeilen allgemein.*
2. *Überlege dir, welche Auswirkung die Ausführung des gesamten Algorithmus hat. In wie weit sind wir unserem Ziel nähergekommen, eine Highscoreliste zu erzeugen? Lade das Programm `alg14b_highscoretabelle` und starte die Animation (siehe Hinweis im Quelltext). Kontrolliere, ob deine Vorhersagen richtig waren.*
3. *Probiere aus, wie oft du die Animation starten musst, bis du eine korrekte Highscore-Tabelle hast. Trifft das nur für diesen Fall zu oder würde die Anzahl der Durchgänge auch bei anderen Punktzahlen ausreichen? Überlege dir, wie viele Durchgänge du maximal im schlechtesten Fall brauchen würdest.*
4. *Implementiere den sogenannten "Bubblesort", indem du mit Hilfe einer Schleife dafür sorgst, dass genügend Durchgänge durchgeführt werden. Kommentiere die Methode aussagekräftig.*